

Uniswap V4 Fee Controller Diff Audit



Uniswap

June 18, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Security Model and Trust Assumptions	6
Low Severity	7
L-01 Hook Self-Reported Fee Classification Can Flip Per Block and Latch Into PoolManager via Permissionless Trigger	7
L-02 Inconsistent Reserved-Family-ID Enforcement Across Configuration Entry Points	8
L-03 Newly Deployed V4FeePolicy Returns Zero Protocol Fee Until Configured, Which the Adapter's Permissionless Trigger Can Latch Into PoolManager [out-of-scope]	9
L-04 Batched V4FeeAdapter.collect Can Be Griefed In-Transaction Via ERC20 Reentrancy Poisoning syncedCurrency [out-of-scope]	9
Notes & Additional Information	10
N-01 HookFeeFlags.USES_DYNAMIC_FEE Bit Name Implies a Pool Property That the Contract Does Not Enforce	10
N-02 CUSTOM_ACCOUNTING_MASK Captures Liquidity-Event Permission Bits Outside the Swap Path [out-of-scope]	11
N-03 Sort-Invariant Seed in setFlagRules Is Vastly Wider Than Any Reachable Popcount [out-of-scope]	11
N-04 Flag-Rule Matching Outcome Depends on Array Order for Rules with Equal Popcount [out-of-scope]	12
Conclusion	13
Appendix	14
Issue Classification	14

Summary

Type	DeFi	Total Issues	8 (3 resolved)
Timeline	From 2026-06-01 To 2026-06-02	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	4 (2 resolved)
		Notes & Additional Information	4 (1 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin audited the diff to the V4 fee controller in the [Uniswap/protocol-fees](#) repository, covering the changes from commit [5eda399](#) to commit [0617abb](#). As an extension of this engagement, OpenZeppelin additionally reviewed a follow-up change to the same files, covering the [diff](#) from commit [31aa2a0](#) to commit [430d89c](#), which builds on the fix-review state of the original audit.

In scope were the following files:

```
src
├── feeAdapters
│   └── V4FeePolicy.sol
└── interfaces
    └── IV4FeePolicy.sol
```

System Overview

The Uniswap V4 fee controller determines the protocol fee applied to each Uniswap V4 pool. `V4FeeAdapter` is the long-lived contract registered as the PoolManager protocol fee controller. It resolves a fee for a given pool, writes it to the PoolManager, and forwards protocol fees accrued in the PoolManager to a designated TokenJar address. `V4FeePolicy` is the replaceable component that computes a pool's protocol fee from automated hook classification and governance-configured parameters. The changes under review are confined to `V4FeePolicy` and its interface.

The in-scope changes restructure how `V4FeePolicy` resolves a pool to a fee. Resolution now assigns the pool a single family identifier and then applies family-specific pricing, replacing the previous separation between a static native-math path and a classified path. A pool's family is derived, in order, from a governance hook assignment, a native-math default applied to static pools (i.e., pools with no return-delta hook bits and a non-dynamic fee), and a set of governance-configured flag rules matched against a hook's self-reported behavioral flags. Pools that do not match any of these conditions are treated as unclassified.

Pricing for a resolved family is drawn first from a per-pair override and then from family-specific logic: native-math pools use a piecewise-linear fee-bucket schedule keyed on the pool's LP fee, while governance families fall back to a per-family default and then to a global default fee. Unclassified pools return the global default fee directly.

The refactor designates two reserved family identifiers that are not ordinary governance families: an unclassified identifier and a native-math identifier. Fee updates reach the PoolManager only through `V4FeeAdapter`, whose triggering and collection entry points are permissionless, while the policy itself is a read-only computation consumed by the adapter and by off-chain tooling.

The follow-up change refines how the native-math default is applied to static pools. Previously, any static pool was assigned the native-math family automatically, whether or not it carried a hook. The change makes this automatic assignment unconditional only for hookless pools; a static pool that carries a hook now joins the native-math family only when a new governance switch, `isHookedNativeMathFeeOn`, is enabled. While the switch is disabled, which is its default, such hooked static pools fall through to flag-rule classification and, absent a match, to the global default fee. The switch is set through a new fee-setter-only entry point.

Security Model and Trust Assumptions

Fee Setter: Configures the policy, including hook family assignments, flag rules, the fee-bucket schedule, per-family default fees, per-pair class fees, and the global default fee, as well as the adapter's per-pool overrides. The fee setter is trusted to configure consistent and intended parameters. Protocol fees are bounded by the PoolManager's maximum protocol fee, so the magnitude of any misconfiguration is capped. Within that bound, the configuration entry points do not all reject the reserved family identifiers, so the fee setter is also relied upon not to assign a reserved sentinel through an entry point that treats it as an ordinary family.

Owner: Sets the adapter's fee setter and replaces the policy contract referenced by the adapter. The owner is trusted to install a correct fee setter and a correct policy. Replacing the policy changes fee computation for every pool at once.

Hooks and Self-Reported Classification: For a pool with no governance family assignment, classification can depend on flags the pool's hook self-reports at resolution time. This value is read fresh on each call and is not pinned, and the adapter's trigger is permissionless, so a hook that varies its self-report across blocks can influence which family is latched into the PoolManager for the pools it serves. Pinning a hook's family through governance configuration is the structural mitigation and is assumed to be applied to hooks whose classification is economically significant.

Permissionless Callers: Any address can trigger a fee update for a pool or collect accrued protocol fees. The trigger writes only the value the policy currently returns, and collection always routes funds to the TokenJar, so these entry points do not grant control over funds. They determine the timing at which a policy result is committed on-chain.

Low Severity

L-01 Hook Self-Reported Fee Classification Can Flip Per Block and Latch Into PoolManager via Permissionless Trigger

The `family-resolution staticcall` re-reads `protocolFeeFlags()` on every invocation and does not pin the result. The `staticcall` opcode forbids state changes inside the callee but permits `SLOAD` and block-context reads, so the hook can branch its return on storage or `block.timestamp`. Because the upstream `V4FeeAdapter.triggerFeeUpdate` is permissionless and latches the policy's return into `PoolManager` via `setProtocolFee`, a hook controller can flip the self-report across blocks and pin the favorable family on-chain by triggering at the right moment.

A concrete sequence:

1. At block `N`, the hook controller calls `hook.flip(cheapFlags)` followed by `V4FeeAdapter.triggerFeeUpdate(key)` in the same transaction. The policy reads `cheapFlags`, resolves to the cheap family, and `PoolManager` latches the cheap protocol fee.
2. The hook controller routes their own swap and pays the cheap fee.
3. At block `N+1`, the hook controller calls `hook.flip(expensiveFlags)` and `V4FeeAdapter.triggerFeeUpdate(key)` again. The policy now reads `expensiveFlags`, and `PoolManager` latches the higher fee for subsequent counterparties.

A hook controller can move the resolved family within the rule-defined fee range across blocks and pin the favorable result on-chain through a permissionless caller, reducing protocol-fee revenue on every pool the hook serves for as long as governance has not pinned the hook via `setHookFamily`.

Consider latching the resolved family on first observation in a dedicated governance-controlled mapping, noting that the structural defense lies in `setHookFamily`.

Update: Acknowledged, not resolved. The team stated:

We believe the impact of this is acceptable given the desired flexibility and potentially valid cases where a hook's classification should reasonably change after its initial report. Won't fix.

L-02 Inconsistent Reserved-Family-ID Enforcement Across Configuration Entry Points

`V4FeePolicy` reserves two `uint8` family identifiers as sentinels: `UNCLASSIFIED_FAMILY_ID` (`0`) and `NATIVE_MATH_FAMILY_ID` (`255`). In `computeFee`, family `0` returns `defaultFee` before the `pairClassFees` lookup, and family `255` is priced from the fee buckets. These sentinels are guarded inconsistently: `setFamilyDefault` and `clearFamilyDefault` reject both, `setFlagRules` rejects only `0`, `__setHookFamily` validates nothing, and `__setPairClassFee` and `__clearPairClassFee` check only currency order and fee bounds.

First, a flag rule or hook assignment may target `255`. Because the flag-rule path runs for non-static pools, a dynamic-fee pool can resolve to `255`; with no `pairClassFees` override, `computeFee` feeds `key.fee` (the `DYNAMIC_FEE_FLAG` sentinel `0x800000`, not an LP fee) into `__computeStaticNativeMathFee`, which clamps to `MAX_PROTOCOL_FEE`. The pre-refactor `!key.fee.isDynamicFee()` guard that prevented this no longer applies.

Second, `__setPairClassFee` accepts a `familyId` of `0`, storing an override that `computeFee` never reads, so the write is dead storage that a fee-setter may believe is active.

Consider rejecting both sentinels in `__setHookFamily` and `setFlagRules` (governance families are `1` to `254`), and rejecting only `UNCLASSIFIED_FAMILY_ID` in `__setPairClassFee` and `__clearPairClassFee`, since `NATIVE_MATH_FAMILY_ID` is the intended native-math pair-override slot. Consider also guarding the native-math branch of `computeFee` against dynamic-fee keys so the sentinel cannot reach `__computeStaticNativeMathFee`.

Update: Resolved at commit [31aa2a0](#).

L-03 Newly Deployed `V4FeePolicy` Returns Zero Protocol Fee Until Configured, Which the Adapter's Permissionless Trigger Can Latch Into `PoolManager` [out-of-scope]

The `V4FeePolicy` constructor initializes only the `POOL_MANAGER` immutable. The `feeSetter`, `defaultFee`, the fee buckets, and the flag rules all start at the zero value, so `computeFee` returns zero for every pool until governance has called every relevant setter. The upstream `V4FeeAdapter.triggerFeeUpdate` is permissionless and writes the policy's return value into `PoolManager.setProtocolFee`, so anyone can mass-trigger every pool during the configuration window and pin a zero protocol fee on each one until the governance team reconfigures the policy and re-triggers the affected pools.

Consider extending the constructor to accept an initial `feeSetter` so the contract leaves the deploy transaction already authorized for configuration, and documenting in the governance guide that `V4FeeAdapter.setPolicy(newPolicy)` should be the last step of a single atomic batch that deploys, configures, and triggers the new policy.

Update: Resolved at commit [83bd154](#). The team stated:

Addressed by way of further documentation clarifying the expected order of operations for governance deploying a new fee policy.

L-04 Batched `V4FeeAdapter.collect` Can Be Griefed In-Transaction Via ERC20 Reentrancy Poisoning `syncedCurrency` [out-of-scope]

`V4FeeAdapter.collect` is permissionless and sequentially iterates over `CollectParams[]`, calling `POOL_MANAGER.collectProtocolFees(TOKEN_JAR, p.currency, p.amount)` for each entry and emitting `FeesCollected` on success.

However, the `collectProtocolFees` function performs an external ERC20 `transfer`, enabling reentrancy into `PoolManager`. Since `PoolManager.sync(Currency)` is externally callable and updates a transient `syncedCurrency`, a malicious fee token can call `PoolManager.sync(targetCurrency)` during an earlier iteration so that a later `collectProtocolFees` in the same batch reverts with `ProtocolFeeCurrencySynced`. This reverts the full adapter batch, wasting gas and impairing reliable multi-currency collection.

The effect is limited to the current transaction because `syncCurrency` is transient, and it does not enable fee theft, since funds still flow to `TOKEN_JAR`.

Consider making `V4FeeAdapter.collect` resilient to per-currency failures (for example, by wrapping each `collectProtocolFees` call in `try/catch` and continuing) and emitting an explicit failure event for monitoring. Consider also providing a single-currency collection endpoint, or recommending single-currency automation, when untrusted ERC20 fee currencies may be present.

Update: Acknowledged, not resolved. The team stated:

| *Searchers are expected to validate their token selection prior to claim. Won't fix.*

Notes & Additional Information

N-01 `HookFeeFlags.USES_DYNAMIC_FEE` Bit Name Implies a Pool Property That the Contract Does Not Enforce

The `USES_DYNAMIC_FEE` constant is part of the hook's self-reported `protocolFeeFlags()` return and is consulted only via flag-rule matching. The pool's dynamic-fee status is instead read directly from `key.fee` at the step-2 gate of `_resolveFamily` as described in [the reference](#), and `protocolFeeFlags()` takes no `PoolKey` argument, so the same hook returns the same bits across every pool it serves.

Consider renaming the bit to one that does not suggest a pool property (for example, `CLAIMS_DYNAMIC_FEE_LOGIC`), or removing it entirely since `key.fee.isDynamicFee()` is already consulted authoritatively at the step-2 gate and the hook's self-report adds no information the policy can verify.

Update: Resolved at commit [ae55fd9](#).

N-02 `CUSTOM_ACCOUNTING_MASK` Captures Liquidity-Event Permission Bits Outside the Swap Path [out-of-scope]

The `CUSTOM_ACCOUNTING_MASK` in `V4FeePolicy` sets all four `*_RETURNS_DELTA` bits, including the two that `PoolManager` consults only inside `afterModifyLiquidity`. As a result, a pool whose hook can return a delta only on liquidity events falls through the `native-math gate` in `_resolveFamily` into the flag-rule path. Absent a `setHookFamily` override or a matching rule, such a pool resolves to family `0` and pays `defaultFee` instead of the bucket-curve entry that its static `key.fee` would otherwise select, even though the swap path itself is unmodified and `key.fee` is the authoritative LP fee.

Consider narrowing `CUSTOM_ACCOUNTING_MASK` to `0xC` so that hooks with only liquidity-event return-delta bits auto-resolve to `NATIVE_MATH_FAMILY_ID` and receive the bucket-curve entry for `key.fee`. Note that the narrower mask also bypasses the flag-rule path for the redirected population, so if flag-rule auto-classification of liquidity-event-only delta hooks is intended, additionally relax the `native-math gate` to enter the flag-rule scan when any liquidity-event-delta bit is set, preserving flag-rule reachability while still routing genuinely vanilla static pools to the bucket curve. In either case, update the [governance guide's custom-accounting definition](#) so the documented semantics match the chosen code path.

Update: Acknowledged, not resolved. The team stated:

This change would have knock-on effects and doesn't negatively impact any known use case. Won't fix.

N-03 Sort-Invariant Seed in `setFlagRules` Is Vastly Wider Than Any Reachable Popcount [out-of-scope]

The `sort check` initializes `previousSpecificity = type(uint256).max` and compares it against `LibBit.popCount(rule.requiredFlags)`, which is structurally bounded by `256`. Seeding `type(uint256).max` is correct but significantly wider than any possible comparand. There is no behavioral difference for valid inputs.

Consider seeding `previousSpecificity = 256` to tighten the invariant.

Update: Acknowledged, not resolved. The team stated:

| *There is no actual behavioral or memory cost difference here, won't fix.*

N-04 Flag-Rule Matching Outcome Depends on Array Order for Rules with Equal Popcount [out-of-scope]

The `setFlagRules` `sort check` rejects only strictly ascending popcount, so equal-popcount rules are accepted in either order. The `match loop in _resolveFamily` returns the first such rule that matches a hook's reported flags, while the [governance guide's flag-rule instructions](#) state to sort by "decreasing popcount" without clarifying that equal popcount is permitted.

Consider clarifying in the governance guide that "decreasing popcount" is non-strict and that the first array element wins on tie, or enforcing a deterministic secondary sort in `setFlagRules`.

Update: Acknowledged, not resolved. The team stated:

| *This doesn't affect any current or expected use case, won't fix.*

Conclusion

The Uniswap V4 fee controller computes and applies the protocol fee for Uniswap V4 pools, deriving each pool's fee from automated hook classification and governance-configured parameters. OpenZeppelin audited the in-scope changes to the [V4FeePolicy](#) contract and its interface, which unify fee resolution around a single family identifier and introduce reserved native-math and unclassified family sentinels.

The central theme of the findings concerns how these reserved sentinels are handled after the refactor. Because the configuration entry points do not enforce the reserved identifiers consistently, a pool can be routed to the native-math family when its key fee is not a valid input, and a per-pair override can be written to a slot that fee computation never reads. The review also identified that, absent a governance family assignment, a hook's self-reported classification is read at resolution time and can be influenced across blocks through the permissionless trigger, with governance pinning of the hook family being the structural mitigation.

The changes are concentrated, and the protocol fee magnitudes are bounded by the PoolManager's maximum protocol fee, which limits the practical impact of the issues raised. Enforcing the reserved family identifiers consistently across all configuration entry points, and constraining family resolution so that native-math pricing is applied only to pools whose fee input is a valid LP fee, would remove the classes of misconfiguration observed.

OpenZeppelin thanks the Uniswap team for their cooperation throughout this engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.